



Module 2

Digital IC Design Flow Overview (Part 2)

cādence[®]

This module spans 3 hours of lecture time.

Module Objectives

In this module, you will:

- Recognize the different stages of front-end design and verification.
- Recognize the different stages of design implementation.

This module follows on from

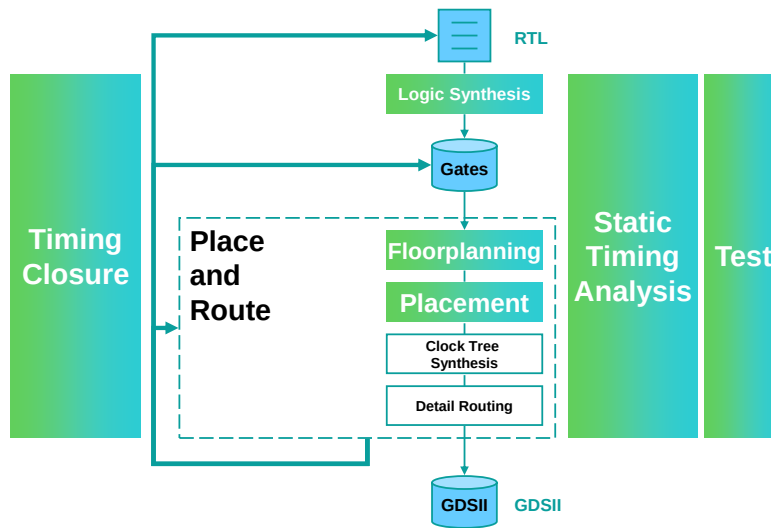
- Digital IC Design Flow Overview (Part 1)



In this module, you will:

- Draw a flow diagram of the entire design flow.
- Identify the distinction between Digital IC design, verification, and implementation.
- Recognize the different stages of front-end design and verification.
- Recognize the different stages of design implementation.
- Identify the challenges of scaling, costs, and physical attributes, as well as low power and area constraints before tapeout.
- Identify the different processes in the semiconductor industry used to handle the above realistic challenges.

Implementation Flow Overview

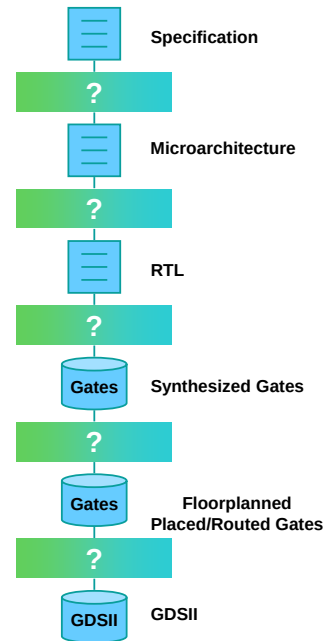


This diagram shows an overview of implementation starting with logic synthesis, which is the process of generating a gate-level netlist from an RTL. The synthesis step is followed by place-and-route. Place-and-route includes floorplanning, placement, clock tree synthesis, and detail routing. Optimizations will be run for setup and hold to meet timing if the timing is not met at the placement, clock tree synthesis, and routing stages. The static timing analysis tool will analyze the timing for violations at each stage, and a report of the violations will be generated. Test structures are added during synthesis to detect manufacturing defects after the parts have been fabricated.

Basic Flow: Implementation

Various representations of logical and physical hardware:

- **Specification:** Textual, graphical, or software representation of the overall behavior and goals of a design.
- **Microarchitecture:** Textual, graphical, or software representation of the initial implementation of the specification.
- **RTL:** Software representation of the design to be implemented.
- **Synthesized gates:** Logical representation of the design after mapping to discrete gates.
- **Placed/routed gates:** Logical and physical representation of the design after place/route.
- **GDSII:** Physical representation of the design just before tapeout.

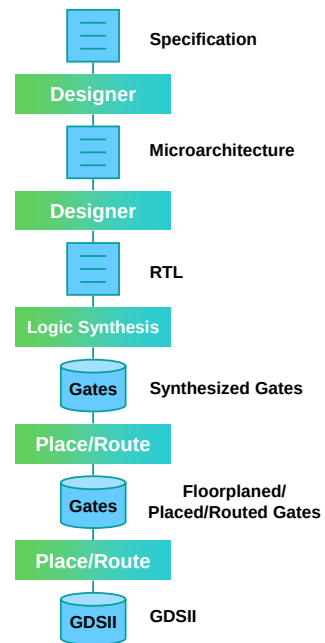


To represent the logical and physical design, create a specification. A specification is the textual, graphical, or software representation of the overall behavior and goals of a design. Also, define the microarchitecture, which is a textual, graphical, or software representation of the initial implementation of the specification. Create an RTL, which is a software representation of the design to be implemented. The RTL will be synthesized into gates which is the logical representation of the design after synthesis maps the netlist to discrete gates. The synthesized gates will be placed and routed into a physical design by the place-and-route tool. After the design is placed and routed, it will be output to a GDS2 file which is the final stage of the implementation process.

Basic Flow: Implementation (continued)

Various resources necessary to translate the representations through the implementation flow:

- **Designer:** The architects of the chip or the block level design engineers.
- **Logic synthesis:** Process of translating RTL into discrete logic.
- **Place/route:** Process of physical design of the discrete logic.



There are several different tasks in implementing a design, and therefore, various types of expertise are necessary to translate the representations through the implementation flow. The design engineers are the architects of the chip, or the block, which will be integrated into a top-level design. There are engineers responsible for logic synthesis, which is the process of translating the RTL into discrete logic.

Engineers responsible for place-and-route create a physical design of the discrete logic.

Implementation Challenges

Implementation

- Size and complexity

The larger and more complex the design, the longer it takes to complete.

- Power

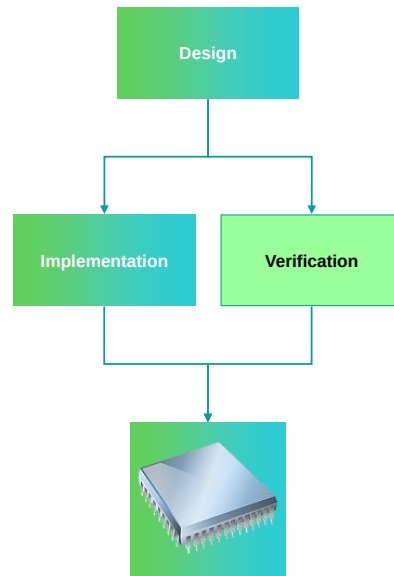
Power must be addressed at many levels during implementation, with very sophisticated strategies to handle the power requirements.

- Timing and design closure

With advanced process technologies, it becomes harder to converge on timing and design rule checks.

- Yield

Affecting the bottom line directly, yield issues must be addressed in the implementation.



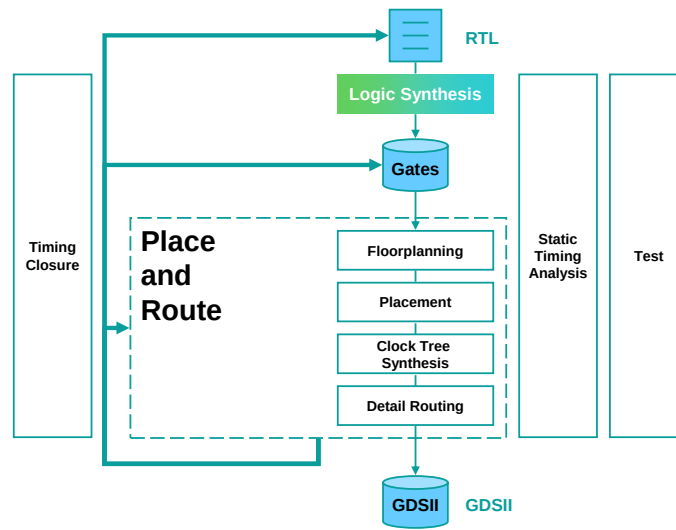
There are several challenges in implementation. The size and complexity of the design are important considerations. The larger and more complex the design, the longer it takes to implement.

If power consumption is an important consideration for the design, power must be addressed at the various stages of implementation, with sophisticated design strategies to handle the power requirements.

Converging timing and fixing design rule violations at advanced process technologies is a constant challenge through implementation.

Yield is measured as the number of working chips or dies on a wafer. To make sure that the parts that are manufactured provide acceptable yields, strategies to maximize yields must be included during implementation.

Logic Synthesis



Logic synthesis is the first step in implementation.

What Is Logic Synthesis?

The process of parsing, translating, and optimizing RTL code into discrete logic gates.

Example: To determine the feasibility of the design, we need to synthesize the RTL code into gates and measure timing, power, and area.

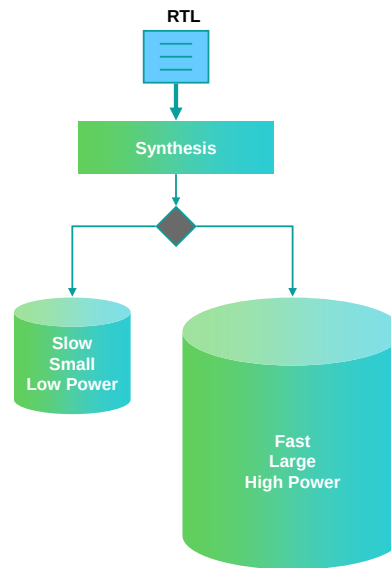


Logic synthesis is where a specification of a design that was written in RTL is converted into a gate-level netlist by the logic synthesis tool. An example of its usage in a sentence is to determine the feasibility of the design; we need to synthesize the RTL into gates and report the timing, power, and area.

How Do We Measure the Quality of the RTL?

By synthesizing to our target technology...

- Timing
 - We can measure the number of logic levels in the design.
 - We can “time” the design against its requirements to see if it meets timing.
- Area
 - We can measure the number of gate units used in the design.
 - We can compare the size of the design against its requirements to see if it meets its area goals.
- Power
 - We can measure the dynamic and leakage power used in the design.
 - We can compare the power numbers against its requirements to see if it meets its power goals.



Logic synthesis generates a gate-level netlist from the RTL and maps it to a target technology.

A few parameters are used to measure the quality of the RTL. They include timing, area, and power.

To measure the quality of the generated netlist, start with the timing. First, you can measure the number of logic levels in the design. Secondly, you can time the design against its constraints to see if it meets timing.

To measure the area, you measure the number of gate units used in the design. And then, compare the size of the design against its requirements to see if it meets its area goals.

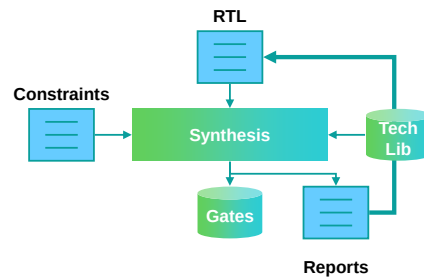
For power, you measure the dynamic and leakage power used in the design. And then, compare the power numbers against its requirements to see if it meets its power goals.

When Do We Change the RTL?

Based on the constraints and the synthesis reports, we will know...

- The timing of the design and which specific paths it violates.
- The overall area of the design.
- The overall power of the design.

If we violate timing, area, or power by a large margin, we *may* have to modify the RTL accordingly.



Sometimes during synthesis, when you check the quality of results, you might need to change the RTL.

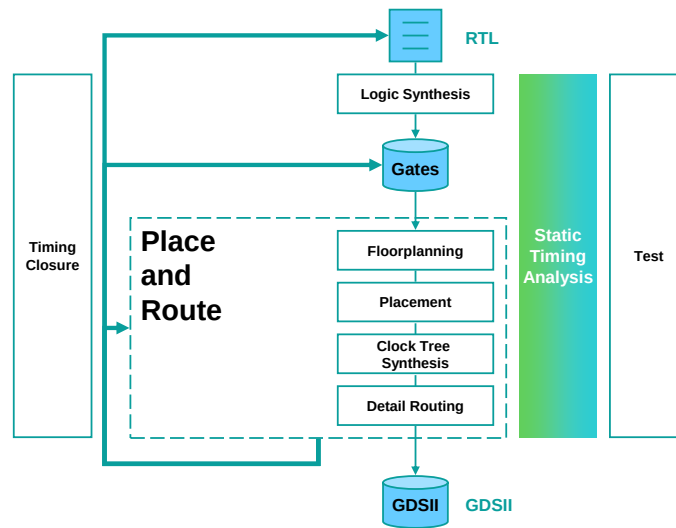
But this depends upon the final results and specifications of the design.

At the end of the synthesis process, you generate various reports and analyze the results based on input constraints and original specifications.

You check for the timing violations, overall area, and power of the design.

If there are some real issues and violations which are not resolvable by modifying the synthesis constraints, then in such scenarios, you might need to modify the RTL.

Static Timing Analysis



11 © Cadence Design Systems, Inc. All rights reserved.



It is important to verify that you are meeting the timing specification during the different stages of the design flow.

As seen here, static timing analysis is used across the board at various steps.

After each design stage, you must constantly run static timing analysis to validate whether your design is meeting your specification, which was previously defined in the form of timing constraints.

What Is Static Timing Analysis or STA?

A method of computing the expected timing of a digital circuit without using SPICE (transistor level) simulation.

- Static timing analysis (STA) is the preferred method for timing signoff.
- After logic synthesis, STA is used to determine if the timing goals of the design are met.
- If the timing goals are not met, then the RTL may have to be modified.
- During the physical design (for example, floorplanning), STA is again used to determine if the timing goals of the design are met.
- If the timing goals are not met, then the RTL may have to be modified.

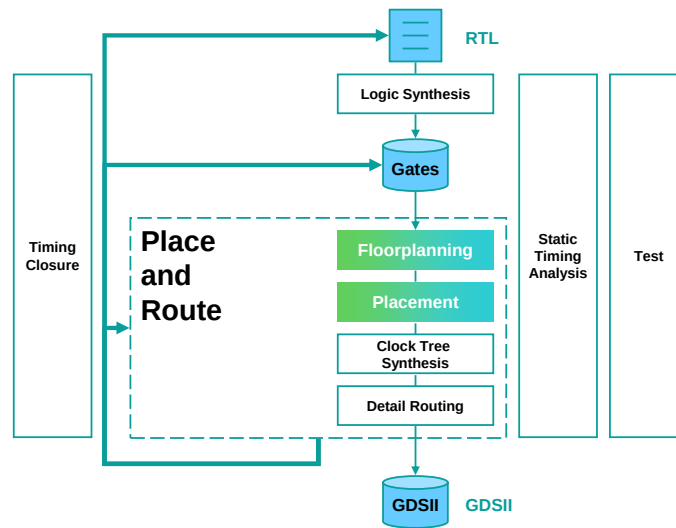
Example: To determine the timing of the design, we ran static timing analysis after synthesis and saw several paths violating their setup time requirements.



Unlike SPICE analysis, which analyzes dynamic waveforms of the signals traveling through the design, static timing analysis uses static tabulated values of timing checks and derived values from timing constraints such as clock periods to compute the required times. Then, it calculates the delay of the path via tabulated values from a timing library and computes arrival times. Finally, static timing analysis determines if a path meets the required time or violates the required time.

If a goal is not met, sometimes, the design is modified using specialized optimization techniques in synthesis. But, if all other workarounds fail, then you might have to go back to modify your RTL.

Floorplanning and Placement



At the top level of the design, floorplanning is the process of laying out the physical partitions of a design to determine the size and connectivity of each partition relative to the chip.

Floorplanning is also where you define the size of the design, place the blocks in the design and plan the power for the design. Placement is the step after floorplanning, where the standard cells, which are discrete components, are placed in the design in specific locations.

What Are Floorplanning and Placement?

Floorplanning is the process of laying out the physical partitions of a design to determine the size and connectivity of each partition relative to the chip.

Floorplanning has a significant effect on the constraints of a design and, therefore, has a significant effect on the RTL.

- The size and the aspect ratio of the block may dictate changes to the RTL.
- RAM sizes, aspect ratios, timing, and power characteristics may dictate changes to the RTL.
- Pin placement may cause the input and output delays to the block to change significantly, requiring attention at the RTL level.

Placement is the process of finding specific locations for each discrete component of an integrated circuit.

Placement has a significant effect on the accurate implementation of the design; therefore, it has a significant effect on the RTL.

- The critical path may be very different between synthesis and placement, so the designer needs to be careful which path in the RTL to fix.
- Because placement (and CTS) can add a significant amount of logic for placement optimization, buffering, etc., the designer needs an accurate area/power estimate to determine if the area/power goals will be met.



Floorplanning has a significant effect on the constraints of a design and, therefore, has a significant effect on the RTL. If the standard cells that were derived from the RTL cannot be placed in the selected size and aspect ratio of the die, this may dictate changes to the RTL.

Additionally, RAM sizes, aspect ratios, timing, and power characteristics may dictate changes to the RTL.

The pin placement may cause the input and output delays to the block to change significantly, requiring attention at the RTL level.

In addition to floorplanning, the placement has a significant effect on the implementation of the design, which in turn may have a significant effect on the RTL.

The critical path may be very different between synthesis and placement, so the designer needs to be careful about which path in the RTL to fix.

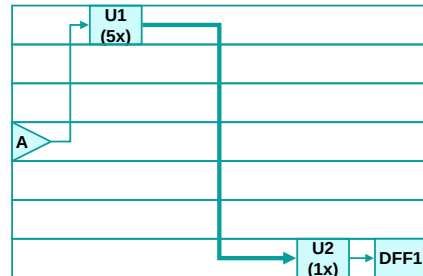
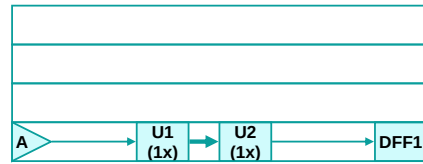
Because placement and clock tree synthesis can add a significant amount of logic for placement optimization and buffering, the designer needs an accurate area and power estimate to determine if the area and power goals will be met.

Example: Placement

In short, placement gives a more accurate picture of the timing and area versus logic synthesis.

Consider the following example:

- Gates U1 and U2 and part of the critical path.
- In logic synthesis, U1 and U2 are “placed” close together, have minimal power and size, and the delay estimated for them is very small.
- In placement, U1 and U2 are placed very far apart, U1 is upsized significantly, so the delay through U1 and through the connection is vastly different.



15 © Cadence Design Systems, Inc. All rights reserved.

cadence

The placement gives a more accurate picture of the timing and area requirements as compared to the estimates at logic synthesis.

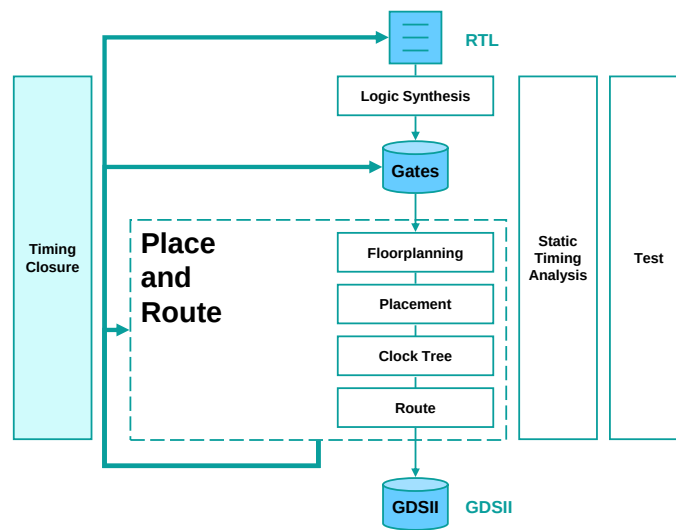
Consider the following example.

Gates U1 and U2 are a part of the critical path.

In logic synthesis, U1 and U2 are timed as being close together, have minimal power and size, and the delay estimated for them is very small.

However, during placement, U1 and U2 are placed very far apart. As a result, U1 is upsized significantly to meet the drive requirements of the larger net delay so that the timing will not be violated.

Timing Closure



After the design is placed and routed, the timing might still not be met for various reasons. Making sure that your design meets the desired specifications is part of the timing closure process.

What Is Timing Closure?

The process of iterating through synthesis, physical design, and verification to converge on the timing of a digital design.

- One of the most challenging aspects of chip design is achieving timing closure.
- Example: To achieve timing closure, the design team went through 20 iterations over a one-month period.



The initial placement of the originally synthesized netlist from the RTL design will be based on an initial design, and further modifications will be necessary.

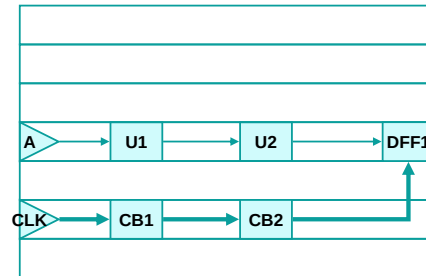
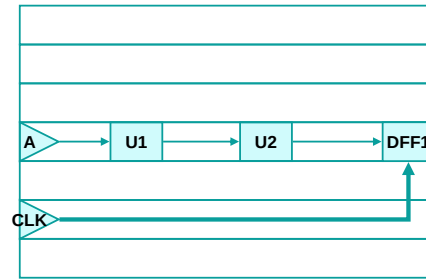
For example, the design will require power planning, test insertion, clock tree synthesis, etc.

These changes affect the timing of the design. To converge the timing of the design, several more modifications to the design must be made, including critical region re-synthesis, redoing some floorplanning, moving the placement of the cells, re-routing certain parts of the modified design, etc.

All these aspects are considered part of timing closure. Timing closure requires multiple loops or iterations.

How Does Timing Closure Affect the Design?

- CTS adds the actual clock buffers, whereas the clocks were “ideal” beforehand.
- Actual clock skew and latency need to be considered for timing.



18 © Cadence Design Systems, Inc. All rights reserved.

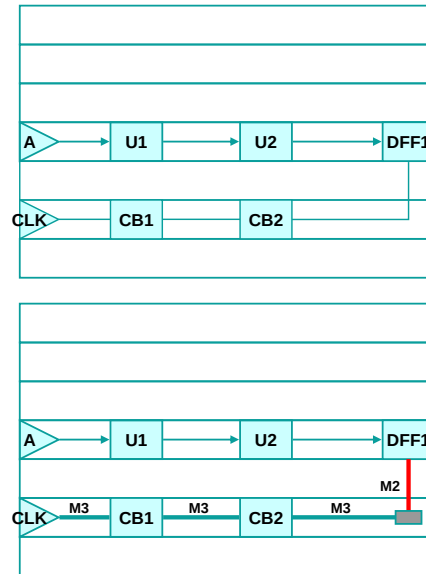


After the clock is routed to the sequential elements in the design, the timing of the design might be far off from what you might have expected. To fix timing, the stage called timing closure will need to implement optimizations to converge or meet timing. Depending on how you implemented the clock tree, it might or might not meet timing in the first round.

How Does Timing Closure Affect the Design? (continued)

Routing adds the actual wire connections, whereas the routing interconnects were estimated beforehand.

- Actual delays with crosstalk can affect timing.
- Congestion can affect routability, which in turn can affect the floorplan, which in turn can affect the overall design constraints greatly.

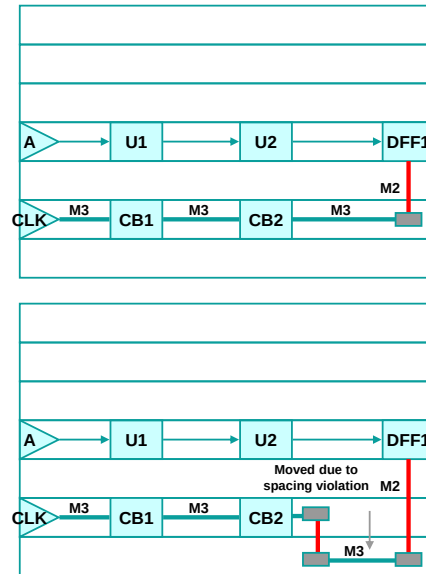


During routing, actual wire connections are added, while the routing interconnects were estimated until this point. Taking into account actual delays with crosstalk can affect timing. Congestion of standard cells in a particular area can affect routability, which can affect the floorplan, and also affect the overall design constraints. In the second diagram, you see that most nets have been routed with metal three, and one net has been routed with Metal 2, which requires a via to connect between the two layers. These actual routes and their associated resistance and capacitances may result in a very different timing report than the reports generated in previous steps, where routes were estimated and not actual.

How Does Timing Closure Affect the Design? (continued)

Physical verification adds yet another dimension since fixing a design rule may break a timing path.

- A spacing violation causes wires to be moved to avoid the violation.
- The wires moving changes the timing and could cause a timing violation.



Physical verification is the process of checking for design rule violations which include open wires or shorted wires. This step adds yet another dimension since fixing a design rule may break a timing path. A spacing violation may result in wires being moved to avoid the violation. The wires that move could change the timing to cause a timing violation.

How Does Timing Closure Affect the RTL? (continued)

CTS, routing, and physical verification can cause changes that affect timing and, therefore, could influence the RTL.

- Changes to the netlist itself or to the placement, CTS, or routing can fix the timing violation without having to modify the RTL.
- In some cases, modifying the RTL is the only choice.
- Modifying the RTL is usually the last resort because it has a very significant impact on the schedule.



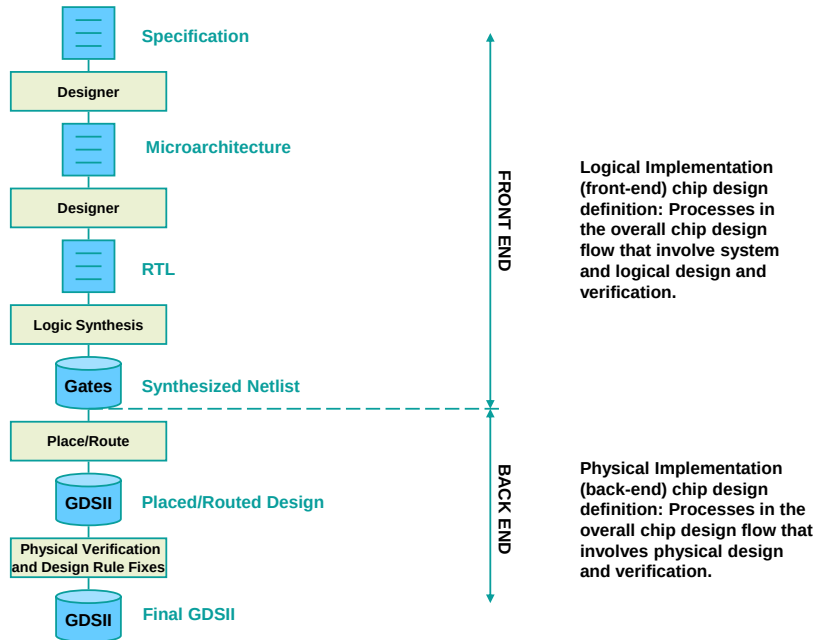
Clock tree synthesis (CTS), routing, and physical verification can cause changes that affect timing and influence the RTL.

Changes to the netlist or to the placement, clock tree synthesis, or routing can fix the timing violation without having to modify the RTL.

In some cases, modifying the RTL is the only choice assuming all other strategies have been tried to no avail.

Modifying the RTL is usually the last resort because it has a very significant impact on the schedule.

Implementation Flow (continued)



22 © Cadence Design Systems, Inc. All rights reserved.

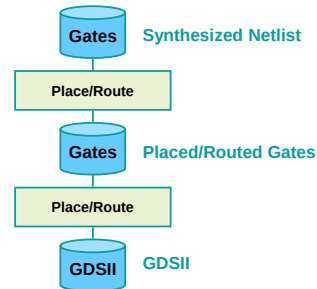


The implementation flow can also be categorized as front-end and back-end. In this particular module, we'll focus mostly on the backend flow or physical implementation.

Back End or Physical Design

The terms “physical design” or “back end” or “place/route” encompass many process steps, such as:

- Floorplanning
- Placement
- Clock Tree Synthesis (CTS)
- Route
- Extraction
- Delay Calculation
- Static Timing Analysis (STA)
- Signal Integrity
- Design Optimization
- Physical Synthesis
- Design Verification
- Mask Prep



These are the main steps in backend or physical design. They include floorplanning, placement, clock tree synthesis (CTS), routing, extraction of the resistances and capacitance in the design, delay calculation to compute the timing, static timing analysis, which compares the existing timing to the constraints and generates a report, signal integrity analysis, design optimization, physical synthesis, design verification, and finally, mask prep.

Terms and Definitions

Floorplanning	Process of deriving the die size, allocating space for soft blocks, planning power, and macro placement.
Placement	Process of placing the standard cells in a floorplanned design.
Clock Tree Synthesis	Process of inserting buffers in the clock tree of a digital design.
Route	Process of connecting the pins of the standard cells, macros, and I/Os of a digital design to specific metal layers in the process technology to match the schematic.
Extraction	Process of calculating the parasitic resistance and capacitance of the interconnect of the physical design.
Delay Calculation	Process of computing the delay of interconnect and standard cells in a digital design.
Static Timing Analysis	Process of computing the timing of logically related paths for a digital design without regard to large-scale functional behavior.
Signal Integrity	Unintended effects on digital signals caused by interconnect parasitic resistance or capacitance that causes noise and/or changes delays.
Design Optimization	Process of using automated algorithms to improve the quality of a digital design.
Physical Synthesis	Process of combining logic synthesis and placement to improve the accuracy of the physical implementation of a digital design.
Design Verification	Process of physically verifying the design rules and backend checks of a design.
Mask Prep	Process of creating the mask set from the GDSII database to allow chip manufacturing.



We need to define many terms for the next slide, which is the overall physical implementation flow we'll cover in the class. Just use this as a reference slide, as we'll cover the details of each step during the lecture.

How Do We Solve the Complex IC Design and Verification Problem?

Divide and Conquer

As we discussed, there are many complex aspects.

- Overwhelmingly complex functional requirements need to be defined correctly and completely in a specification.
- Specification needs to translate into a physical design.
 - This requires multiple levels of abstraction, all of which need to be verified against the original intent.
 - We even need to consider quantum physics effects in the physical layout.
- The physical design needs to be economically manufacturable and testable.

There is almost zero room for error.

- A single mistake is likely to kill the entire project.
- It's a physical entity, not software that can be patched.

A complex IC requires literally hundreds or thousands of hardware engineers to design and verify.



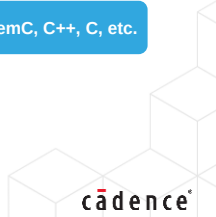
The most effective way to solve complex IC design and verification problems is by divide and conquer. There are many complex aspects to the process, from specifications to implementation. To start with, complex functional requirements need to be defined correctly and completely in a specification. The specification needs to be translatable into a physical design. To translate the spec into a physical design, there are multiple levels of abstraction, all of which need to be verified against the original intent. Because of shrinking geometries, we even need to consider quantum physics effects in the physical layout. Even if it's possible to manufacture the design, the physical design needs to be economically manufacturable and testable. Therefore, there is almost zero room for error. A single unforeseen error can kill the entire project. Since the final output is a physical part, the flaws can be fatal to the project, unlike software that can be patched.

A complex IC can require hundreds or thousands of hardware engineers to design and verify.

What Does Divide and Conquer Mean in This Context?

Each aspect of the Design and Verification is likely to use multiple languages and file formats for one or more:

- Functional concept experimentation and modeling 
- Functional definition 
- Execution 
- Functional verification with simulation/formal proofs/hardware acceleration/emulation 
- Physical description and timing constraints 



Each aspect of the design process requires an understanding of various programming languages and file formats, depending on what aspect of the design process is being worked on. C++, SKILL® and Tcl are examples of programming languages, whereas .lib, GDSII and ECSM are examples of file formats.

How Many Software Tools Do We Need?

There are many separate software tools required in the flow.

All of these are integrated into "platforms" or "suites."

A single IC design and verification flow may not utilize all tools listed.

We will explain these different tools and their purpose as the course progresses.

To give an example of scale, an extract from the www.cadence.com A-Z list of tools.

A	• AI Boost NNE 110 • Allegro ECAD-MCAD Library Creator • Allegro Package Designer Plus Silicon Layout Option • Allegro PCB Symphony Team Design Option • Allegro X AI • AWR Analyst Software • AWR Microwave Office • AWR VSS Software	• AI Max Multi-Core • Allegro EDM Solution • Allegro Package Designer Plus SIP Layout Option • Allegro PCB Symphony Team Design Option for IC Packaging • Allegro X Design Platform • AWR AXIEM Analysis • AWR Software Options	• AI Max NNA 110 • Allegro Package Designer Plus • Allegro PCB Designer • Allegro PCB Librarian • Allegro Pulse • Allegro System Capture • Assura Physical Verification • AWR Design Environment Platform • AWR TX-Line
C	• Cadence Cerebrus Intelligent Chip Explorer • Celsius EC Solver • Certus Closure Solution • Clarity • Clarity 3D Transient Solver • Clarity PCB Extraction • Conformal Equivalence Checker • Conformal Overview • ConnX B10/B20 • Controller for PCIe	• Cadence Specman Elite • Celsius • Celsius PowerDC • Circuit Design • Clarity 3D Solver • Clarity Advanced IC Packaging Extraction • Compute Mesh Spacing for a Given Y+ for Viscous Flow in CFD • Conformal Litmus • Conformal Smart LEC • ConnX DSPs • Controller for PCIe and CXL	• Cadence Verification • Celsius Advanced PTI • Celsius Thermal Solver • Circuit Simulation • Clarity 3D Solver Cloud • Clarity Extraction • Clarity IC Packaging Extraction • Conformal Constraint Designer • Conformal ECO Designer • Conformal Low Power • ConnX 110/120 • Controller for CXL • Custom Design Migration
D	• DAM Media Carousel Test • Design IP • Virtuoso Digital Implementation • Virtuoso Integrated Physical Verification System • Virtuoso System Design Platform • Vision P1 • Vision Q8 • Voltus-XFI Custom Power Integrity Solution	• DDR/LPDDR PHY and Controller • Virtuoso Heterogeneous Integration • Virtuoso Layout Suite • Virtuoso Schematic Editor • Virtuoso Variation Option • Vision P6 • Voltus IC Power Integrity Solution	• DDR5, DDR4, DDR3 PHY and Controller • Virtuoso InDesign DRC • Virtuoso RF Solution • Virtuoso Studio • Vision DSPs • Vision Q7 • Voltus-Fi Custom Power Integrity Solution
X	• Xcelium Logic Simulator • Xtensa NX Processor Platform	• xSPI	• Xtensa LX Processor Platform

There are many separate software tools required in the flow. All of these are grouped into platforms based on the stage where they are used in the design process. A single IC design and verification flow may not utilize all tools listed. We will explain these different tools and their purpose as the course progresses.

Expect to Specialize

No individual can be at an expert level in every tool and every language.

If one had time to learn everything, then before you finished, you would need to start over again.

- Like painting the Sydney Harbour Bridge.

For a career in the IC design verification industry:

- Expect to specialize in a maximum of 3-4 languages/tools at each stage of your career.
- Expect to take 2 years to get good at it.
- Expect to constantly renew your knowledge or face becoming obsolete.
 - After any 5-year window, technology and the how things are done has almost completely changed.



No individual can be at an expert level in every tool and every language. If one had time to learn everything, then before you finished, you would need to start over again. It's a bit like painting the Sydney Harbour Bridge. For a career in the IC design verification industry, expect to specialize in a maximum of 3-4 languages/tools at each stage of your career. Expect to take 2 years to get good at it. Expect to constantly renew your knowledge by expecting major changes every five years, along with frequent incremental changes.

Implementation Summary

- The implementation flow starts with RTL and progresses through a series of varying representations.
- There are many things to consider when coding RTL, both on the front end (from specification to microarchitecture) and the back end, through synthesis and place/route.
- As the design progresses through the flow, more accurate representations of the design are created, thus more accurate measures of timing, area, and power.
- RTL changes during this process to ensure that the design meets its overall requirements and specifications.
- Changes to the RTL very late in the process impact the schedule significantly.



The implementation flow starts with RTL and progresses through a series of varying steps.

There are many factors to consider when coding RTL. On the front end, factors include specification and microarchitecture. On the back end, factors include through synthesis and place/route. As the design progresses through the flow, more accurate representations of the design are created, thus, more accurate measures of timing, area, and power are generated. The RTL changes during this process to ensure that the design meets its overall requirements and specifications. Changes to the RTL very late in the process impact the schedule significantly.

Summarizing So Far...

We have introduced all of the steps in the IC Design and Implementation flow:

- Specification, microarchitecture, RTL, logic synthesis
- Design verification

We have also explained how and why one has to gain expertise in only one of these streams, not in both, either *Design and/or Verification or Implementation*.



This page does not contain notes.